Step 8 — Merge, export, soglia, (opzionale) distillazione

Obiettivo

Trasformare l'adapter scelto in un **fact checker pronto all'uso**: pesi fusi, formato di produzione, una sola domanda congelata, soglia e temperatura definitive, verifica che nulla sia cambiato nel passaggio, congelamento e test finale. In più due opzioni specifiche del compito binario: **estrarre la testa** per servire il modello come classificatore e **distillare** un encoder piccolo.

Input e output

Input	adapter scelto, pesi di partenza, `dev_select`/`dev_confirm`, `finetune/question.json`
Output	`finetune/release/<modello>-fc-<versione>/`: pesi fusi bf16, (Winnow) GGUF, `decision.json`, `frozen.json`, report, model card; opzionali `classifier/` e `student/`
Codice	`finetune/merge.py`, `finetune/extract_head.py`, `finetune/distill.py` (**da scrivere**)

1. Fusione e verifica

Identiche al piano generale (../fine_tuning/08_merge_export.md §1–3): `merge_and_unload()` su pesi **bf16** (anche se il training era QLoRA); copia di tokenizer e template originali; confronto PEFT contro fuso su 200 righe di `dev_select` (differenza media di $P < 0{,}005$, al massimo 1 decisione diversa). Per Winnow: conversione GGUF BF16 → Q8_0 e controllo sui 63 esempi contro vLLM.

2. Una sola domanda in produzione

`decision.json` descrive esattamente come chiamare il modello:

```
{"model": "winnow-12b-fc-v1", "format": "gguf-q8_0",
 "question": "support_simple", "question_text": "Does `document` support `claim`? ...",
 "options": {"A": "false", "B": "true"}, "read": "P(B)",
 "temperature": 1.0, "threshold": 0.50,
 "chunking": {"max_tokens": 8192, "aggregate": "max"},
 "fitted_on": "dev_select", "confirmed_on": "dev_confirm"}
```

(Numeri di esempio, non risultati.) Qualsiasi chiamante che usi un'altra formulazione esce dalle condizioni valutate.

3. Soglia e temperatura finali

Come nel piano generale: temperatura (log-loss) poi soglia (BAcc macro) su `dev_select`, conferma su `dev_confirm`, soglia secondaria dichiarata sul dev di AggreFact. In più: se la BAcc a soglia 0,5 è entro 0,5 punti da quella fittata, **in produzione si usa 0,5** (più robusta ai cambi di dominio, come MiniCheck) e la soglia fittata si riporta solo nel report.

4. Opzione: estrarre la testa binaria

Il fact checker legge solo `σ(u · h)` con `u = (w_vero − w_falso) / T` (00a §4). Si può salvare il modello come classificatore:

```
from transformers import AutoModelForCausalLM
import torch

lm = AutoModelForCausalLM.from_pretrained(MERGED_DIR, torch_dtype=torch.bfloat16)
```

```
W = lm.get_output_embeddings().weight            # [V, d]
u = (W[TRUE_ID] - W[FALSE_ID]) / TEMPERATURE     # [d]
# testa a 2 classi equivalente: logit [0, u·h]  (oppure [-u·h/2, u·h/2])
score = torch.stack([torch.zeros_like(u), u])    # [2, d], riga 1 = "supportato"
torch.save({"score.weight": score}, f"{OUT_DIR}/score.pt")
# poi: config con architettura *ForSequenceClassification, num_labels = 2,
# pesi del corpo identici, "score.weight" = score
```

Vantaggio	Attenzione
Niente proiezione sul vocabolario (~260k righe per Winnow): meno memoria e calcolo per l'ultimo passo	serve che l'architettura abbia una classe `ForSequenceClassification` supportata dal server (vLLM la supporta per molte architetture decoder; per `gemma4_text` e per i layer DeltaNet di Qwen3.5 va verificato)
Il server restituisce direttamente una probabilità	il classificatore legge l'**ultimo** token: il prompt deve finire esattamente con `Answer:` come in training eventuale softcapping/normalizzazione dei logit va riprodotto; verifica su 200 righe contro il modello generativo (stessa soglia di §1)

Se il server non supporta la classe, non si perde nulla: si continua a leggere i logprob delle due lettere (1 token generato, `logprobs` sulle due lettere), come nei benchmark della repo.

5. Opzione: distillazione in un encoder

Solo se il costo d'inferenza lo richiede (fase S di 07).

1. **Dati per lo studente**: tutte le righe del mix (stadi 1 e 2) più un insieme grande non etichettato a mano (VitaminC, FEVER, HoVer interi, coppie D2C non verificate), tutte dal **solo train**.
2. **Etichette dell'insegnante**: P(supportato) del modello finale congelato, con la stessa domanda e lo stesso chunking dell'encoder (chunk da ~550 token, così insegnante e studente vedono lo stesso testo).
3. **Training dello studente** (configurazione in 05 §9): `L = KL(p_insegnante || p_studente) + 0,5 · CE(etichetta vera)` dove l'etichetta vera esiste.
4. **Valutazione** con la stessa suite; obiettivo indicativo: studente $\geq$ FactCG-DBT (75,6) e a $\leq$ 1,5 punti dall'insegnante sulla media dei blocchi.

6. Congelamento e test

`frozen.json` prima del test: hash dei pesi (o del GGUF), del tokenizer e del template, di `question.json`, dei mix e dei dataset renderizzati, configurazione di training, `decision.json`, hash del codice di valutazione. Poi una sola esecuzione dei test (06).

7. Pacchetto finale e licenze

File	Contenuto
pesi (safetensors e/o GGUF)	modello fuso
`decision.json`	domanda, lettere, soglia, temperatura, chunking
`frozen.json`	hash e provenienza
`REPORT.md`	risultati, confronto con FactCG/MiniCheck/Bespoke, limiti
`MODEL_CARD.md`	uso previsto: verifica binaria documento–claim in inglese, con documento fornito. **Non** è più un decision model generale: le altre domande non sono valutate

Licenze: come nel piano generale. JPT-9B CC BY-NC 4.0 (anche il derivato); Winnow Apache 2.0 ma il derivato eredita i vincoli dei dati (ANLI, FactKG, AVeriTeC, SciFact non commerciali; MiniCheck e FactCG MIT; dati generati con modelli aperti senza vincoli). Lo studente distillato eredita i vincoli dell'insegnante e dei dati.

Criteri di completamento

- Verifica della fusione (e del GGUF per Winnow) superata.
- Soglia e temperatura su `dev_select`, confermate su `dev_confirm`; decisione 0,5 vs fittata documentata.
- (Se fatta) testa estratta verificata contro il modello generativo.
- `frozen.json` prima del test; test una volta; report e model card.